Multiple regression part 3: prediction and fit

Frank Edwards

Data for today

```
dat <- read_csv("https://www.openintro.org/data/csv/acs12.csv")
glimpse(dat)
```

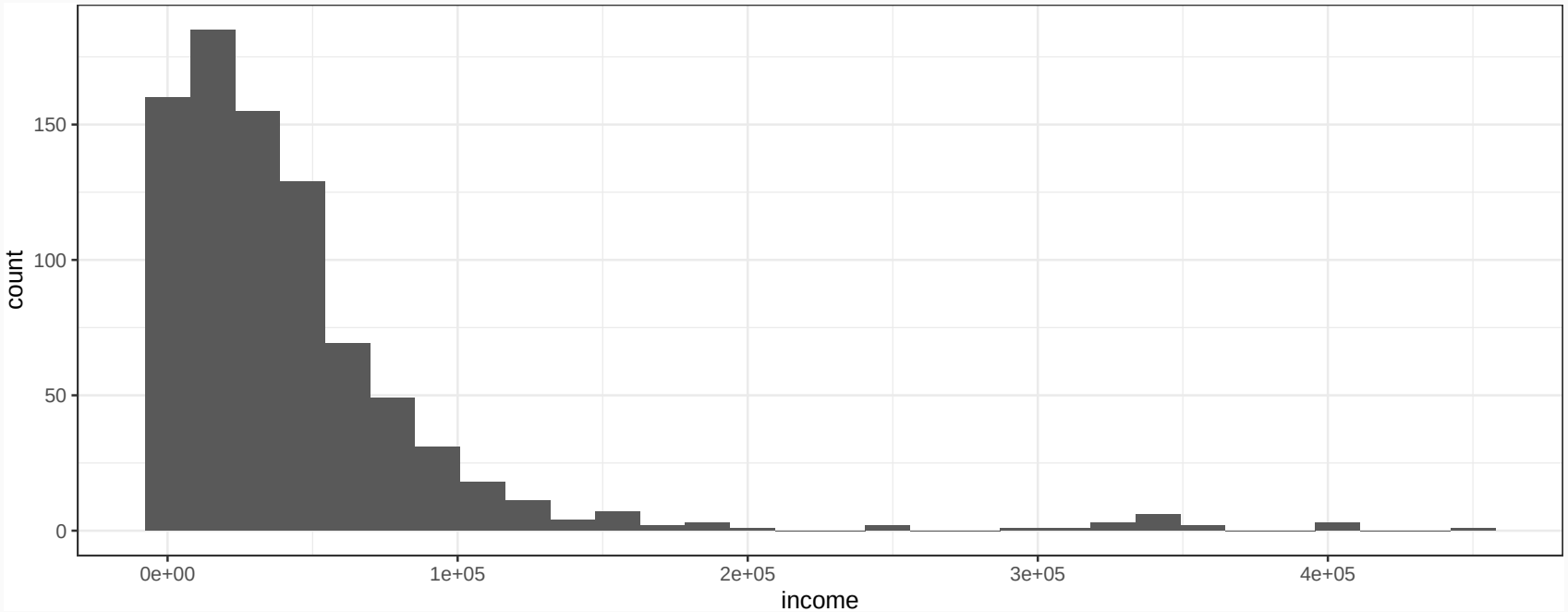
```
Rows: 2,000
Columns: 13
$ income       <dbl> 60000, 0, NA, 0, 0, 1700, NA, NA, NA, 45000, NA, 8600, 0,~
$ employment   <chr> "not in labor force", "not in labor force", NA, "not in l~
$ hrs_work     <dbl> 40, NA, NA, NA, NA, 40, NA, NA, NA, 84, NA, 23, NA, NA, N~
$ race         <chr> "white", "white", "white", "white", "white", "other", "wh~
$ age          <dbl> 68, 88, 12, 17, 77, 35, 11, 7, 6, 27, 8, 69, 69, 17, 10, ~
$ gender       <chr> "female", "male", "female", "male", "female", "female", "~
$ citizen      <chr> "yes", "yes", "yes", "yes", "yes", "yes", "yes", "yes", "~
$ time_to_work <dbl> NA, NA, NA, NA, NA, 15, NA, NA, NA, 40, NA, 5, NA, NA, NA~
$ lang         <chr> "english", "english", "english", "other", "other", "other~
$ married      <chr> "no", "no", "no", "no", "no", "yes", "no", "no", "no", "y~
$ edu          <chr> "college", "hs or lower", "hs or lower", "hs or lower", "~
$ disability   <chr> "no", "yes", "no", "no", "yes", "yes", "no", "yes", "no",~
$ birth_qrtr   <chr> "jul thru sep", "jan thru mar", "oct thru dec", "oct thru~
```

Subset the data to only include workers

```
dat <- dat |>
    filter(employment == "employed")
```

Let’s look at income for this ACS 2012 sample

```
ggplot(dat, aes(x = income)) + geom_histogram()
```



Multiple regression (regression with more than 1 predictor)

We can generalize the linear regression

$$Y = \beta_0 + \beta_1 X + \varepsilon$$
$$\varepsilon \sim N(0, \sigma^2)$$

Multiple regression (regression with more than 1 predictor)

We can generalize the linear regression

$$Y = \beta_0 + \beta_1 X + \varepsilon$$
$$\varepsilon \sim N(0, \sigma^2)$$

as

$$Y = \beta_0 + \beta_1 x_1 \cdots \beta_k x_k + \varepsilon$$
$$\varepsilon \sim N(0, \sigma^2)$$

Where k is the number of predictor variables we include in the model. Our only constraint is that k must be smaller than the number of observations n in our data.

Multiple regression basics

1. Categorical predictors act as intercepts, or differences in level
2. Continuous predictors act as slopes

How to assess a model

If our goal is causal inference

1. Construct causal theories (DAGs)
2. Determine testable implications of model
3. Evaluate, critique, refine

If our goal is prediction

1. Establish what we'd like to predict
2. Estimate competing models
3. Compare with metric of accuracy / fit

Fit a model for prediction

Let's start with an additive model

$$E(i) = \beta_0 + \beta_1 age + \beta_2 gender + \beta_3 education$$

```
m1 <- lm(income ~ age + gender + edu, data = dat)
```

The most common measure of model fit: R^2

$$R^2 = \frac{var(\text{outcome}) - var(\text{residuals})}{var(\text{outcome})}$$

$$R^2 = 1 - \frac{var(\text{residuals})}{var(\text{outcome})}$$

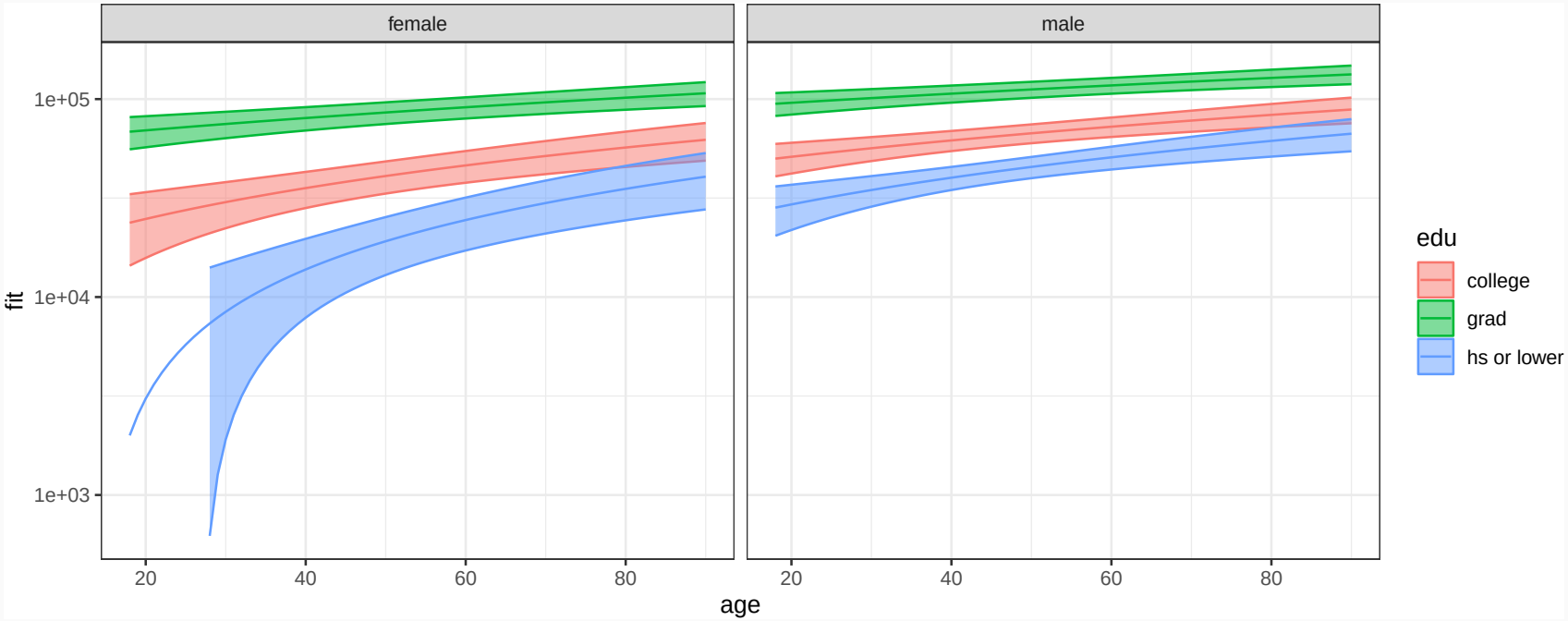
To understand what the model says, let's visualize y-hat

```r
# set up prediction data with values of interest
age <- 18:90
edu <- c("college", "hs or lower", "grad")
gender <- unique(dat$gender)
pred_dat <- expand_grid(age, edu, gender)
# generate expected values and CI, join pred_dat
e_y <- predict(m1, newdata = pred_dat, interval = "confidence") |>
    bind_cols(pred_dat)
# inspect
head(e_y)
```

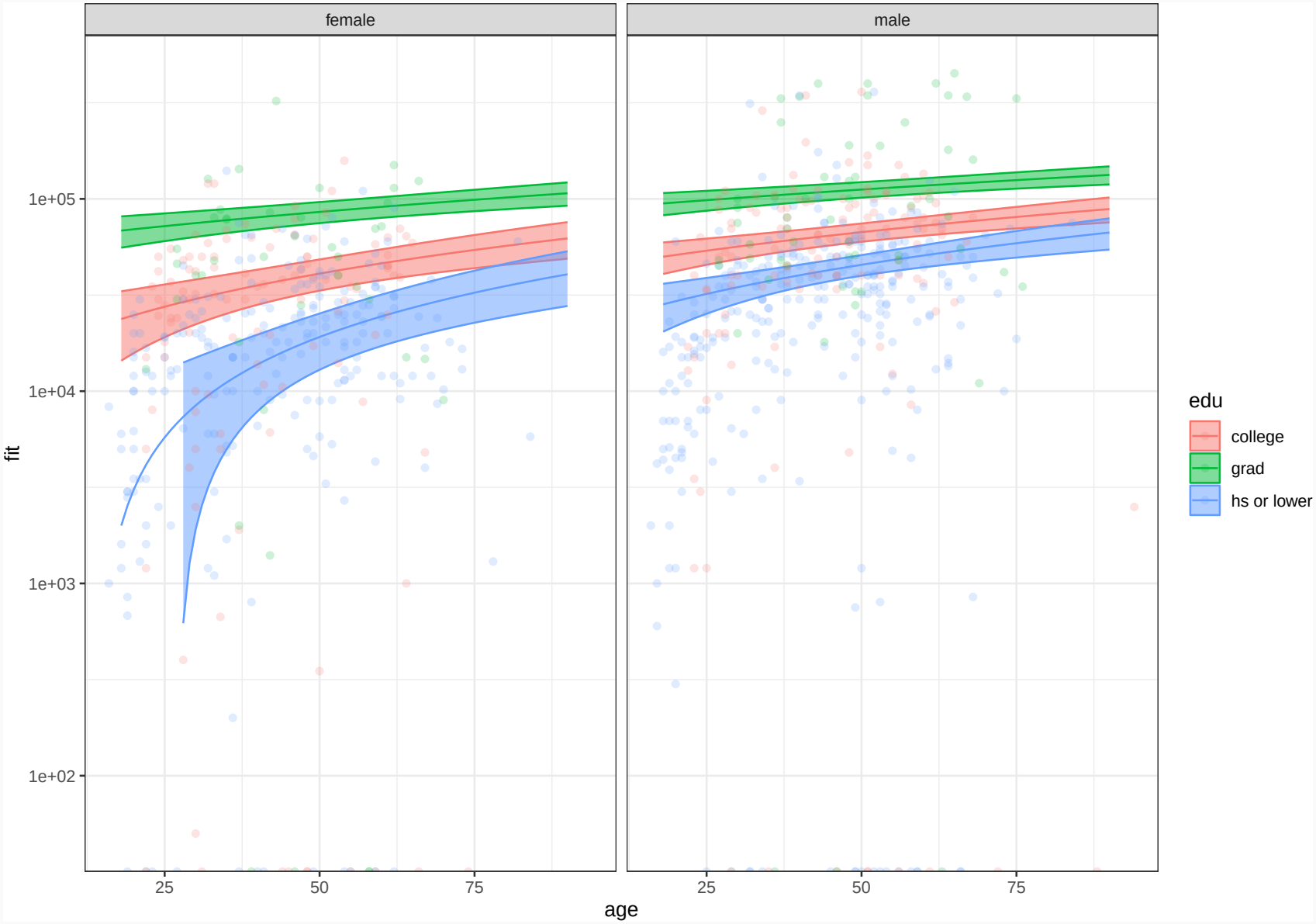
```
# A tibble: 6 x 6
     fit    lwr     upr   age edu         gender
   <dbl>  <dbl>   <dbl> <int> <chr>       <chr>
1 23736. 14406.  33066.    18 college     female
2 50032. 40691.  59373.    18 college     male
3  1999. -6139.  10137.    18 hs or lower female
4 28295. 20395.  36195.    18 hs or lower male
5 68432. 55750.  81115.    18 grad        female
6 94728. 82201. 107255.    18 grad        male
```

Visualizing model expectations

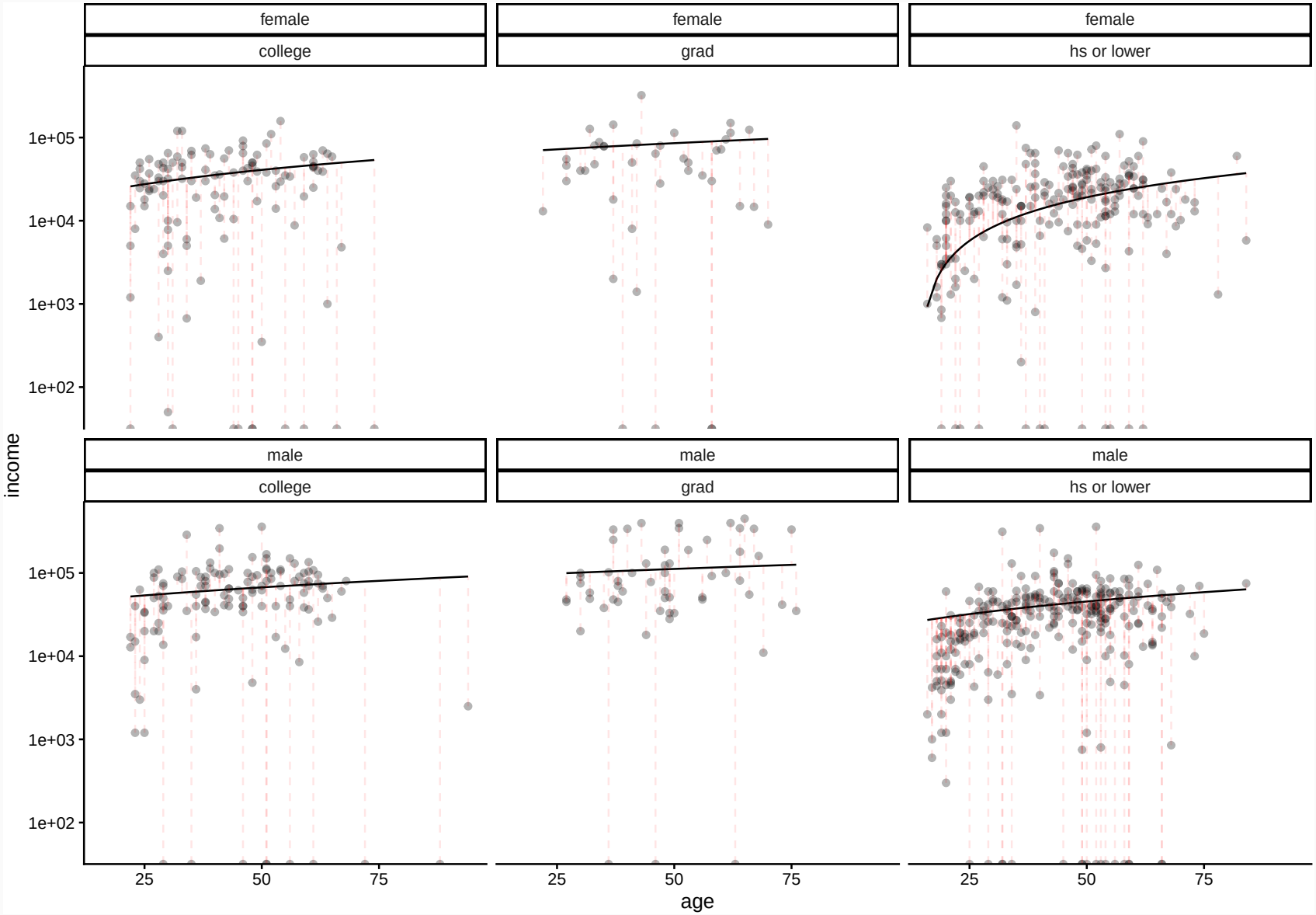
```
ggplot(e_y, aes(y = fit, ymin = lwr, ymax = upr, x = age, fill = edu, color = edu)) +
    geom_ribbon(alpha = 0.5) + geom_line() + facet_wrap(~gender) + scale_y_log10()
```



How well does the model fit?



How well does the model fit?



Let's check the magnitude of error for each category

We'll compute the average of the squared error (residual) to ensure that negative and positive errors don't cancel each other. Which group has the largest average (squared) error?

```
resid_plot |>
    group_by(edu, gender) |>
    summarize(resid2 = mean(resid^2)) |>
    arrange(resid2)
```

```
# A tibble: 6 x 3
# Groups:   edu [3]
  edu          gender       resid2
  <chr>        <chr>         <dbl>
1 hs or lower female   409557456.
2 college     female   840303287.
3 hs or lower male    1679254053.
4 college     male    3399003503.
5 grad        female  3981468684.
6 grad        male   15268680954.
```

Let's compute a simple fit statistic

$$R^2 = 1 - \frac{var(\text{residuals})}{var(\text{outcome})}$$

```
# compute R^2 manually
1 - var(m1$residuals)/var(dat$income)
```

```
[1] 0.2231234
```

Let's compute a simple fit statistic

$$R^2 = 1 - \frac{var(\text{residuals})}{var(\text{outcome})}$$

```
# compute R^2 manually
1 - var(m1$residuals)/var(dat$income)
```

```
[1] 0.2231234
```

This means that our model explains 22 percent of the variation in income.

Not bad!

We can also get it from summary()

```
summary(m1)
```

```
Call:
lm(formula = income ~ age + gender + edu, data = dat)

Residuals:
    Min      1Q  Median      3Q     Max
-118821  -23579   -4034   11908  330108

Coefficients:
               Estimate Std. Error t value Pr(>|t|)
(Intercept)     14098.5     6325.0   2.229   0.0261 *
age               535.4      120.3   4.450 9.76e-06 ***
gendermale      26296.1     3504.2   7.504 1.58e-13 ***
edugrad         44696.5     6049.3   7.389 3.59e-13 ***
eduhs or lower -21736.8     3957.0  -5.493 5.24e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 50460 on 838 degrees of freedom
Multiple R-squared:  0.2231,    Adjusted R-squared:  0.2194
F-statistic: 60.17 on 4 and 838 DF,  p-value: < 2.2e-16
```

Can we improve the fit?

When our goal is *prediction*, we are often interested in improving how well the model fits. Confounding and causal inference are not primary here.

Can we improve the fit?

When our goal is *prediction*, we are often interested in improving how well the model fits. Confounding and causal inference are not primary here.

Let's add hours worked as a predictor

```
m2 <- lm(income ~ age + gender + edu + hrs_work, data = dat)
# pull r2 from summary()
summary(m2)$r.squared
```

```
[1] 0.2728331
```

Can we improve the fit?

When our goal is *prediction*, we are often interested in improving how well the model fits. Confounding and causal inference are not primary here.

Let's add hours worked as a predictor

```
m2 <- lm(income ~ age + gender + edu + hrs_work, data = dat)
# pull r2 from summary()
summary(m2)$r.squared
```

```
[1] 0.2728331
```

How should we interpret R^2 here? Have we improved the fit?

More flexible models will improve fit

Let's allow for heterogeneity in intercepts and slopes with interactions

```
m3 <- lm(income ~ age + gender * edu * hrs_work, data = dat)
summary(m3)$r.squared
```

```
[1] 0.3013753
```

Have we improved the fit?

More flexible models will improve fit

```
m4 <- lm(income ~ age * gender * edu * hrs_work, data = dat)
summary(m4)$r.squared
```

```
[1] 0.3240637
```

```
m5 <- lm(income ~ (age + I(age^2)) * gender * edu * hrs_work, data = dat)
summary(m5)$r.squared
```

```
[1] 0.3359174
```

```
m6 <- lm(income ~ (age + I(age^2) + I(age^3)) * gender * edu * hrs_work, data = dat)
summary(m6)$r.squared
```

```
[1] 0.3524448
```

So which is best?

Overfitting

In predictive modeling, we have to balance the desire to accurately predict our sample against the danger of *overfitting*.

Overfitting is when we specify a model that describes our observed data well, but explains new data poorly.

Overfitting in action

```
d <- mtcars %>%
    sample_n(10)
m1 <- lm(mpg ~ wt, data = d)
m2 <- lm(mpg ~ wt + I(wt^2), data = d)
m3 <- lm(mpg ~ wt + I(wt^2) + I(wt^3), data = d)
m4 <- lm(mpg ~ wt + I(wt^2) + I(wt^3) + I(wt^4), data = d)
m5 <- lm(mpg ~ wt + I(wt^2) + I(wt^3) + I(wt^4) + I(wt^5), data = d)
m6 <- lm(mpg ~ wt + I(wt^2) + I(wt^3) + I(wt^4) + I(wt^5) + I(wt^6), data = d)
```

Which model is best?

```
summary(m1)$r.squared
```

```
[1] 0.7801811
```

```
summary(m2)$r.squared
```

```
[1] 0.7867191
```

```
summary(m3)$r.squared
```

```
[1] 0.8592646
```

```
summary(m4)$r.squared
```

```
[1] 0.8674812
```

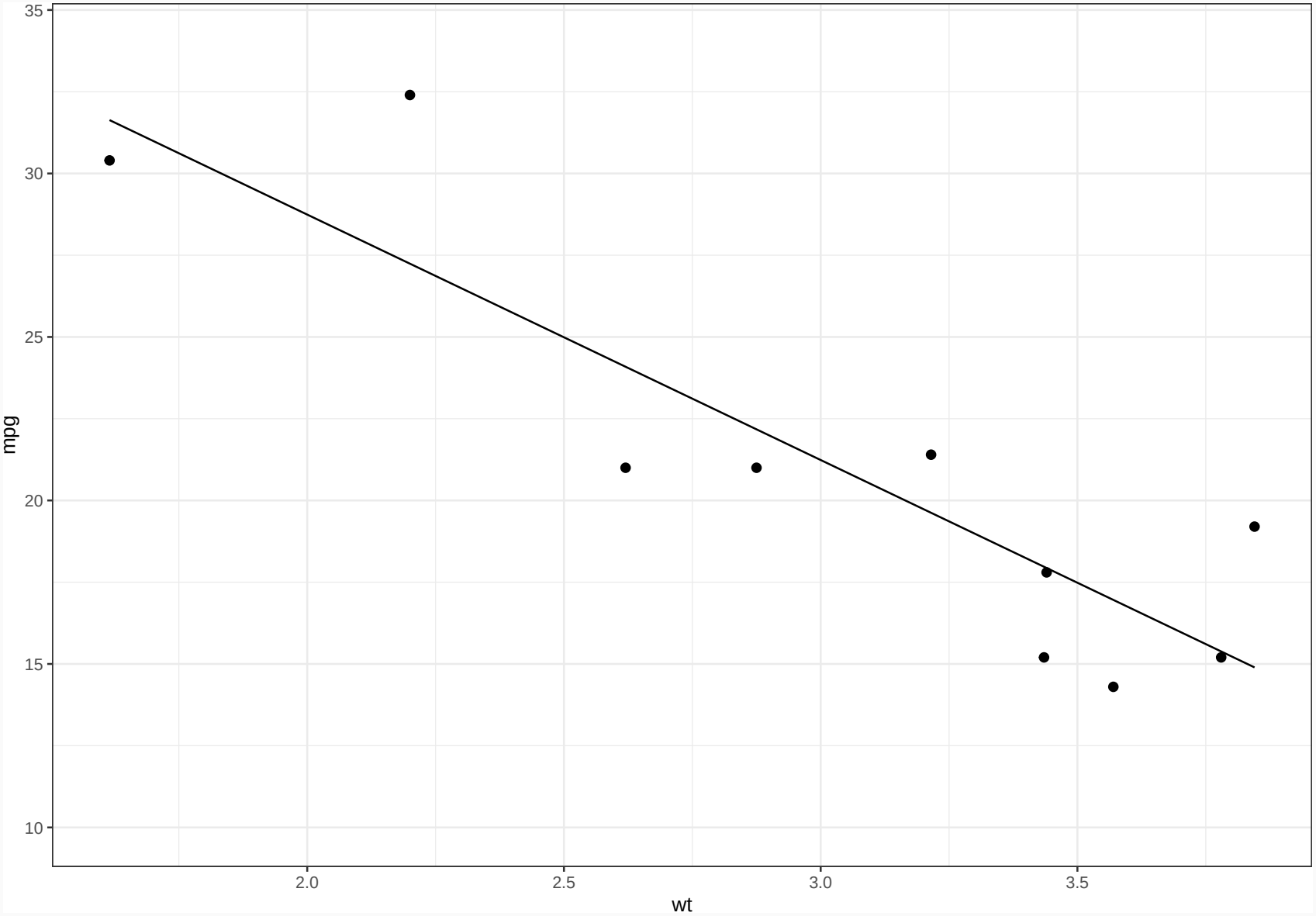
```
summary(m5)$r.squared
```

```
[1] 0.9875247
```

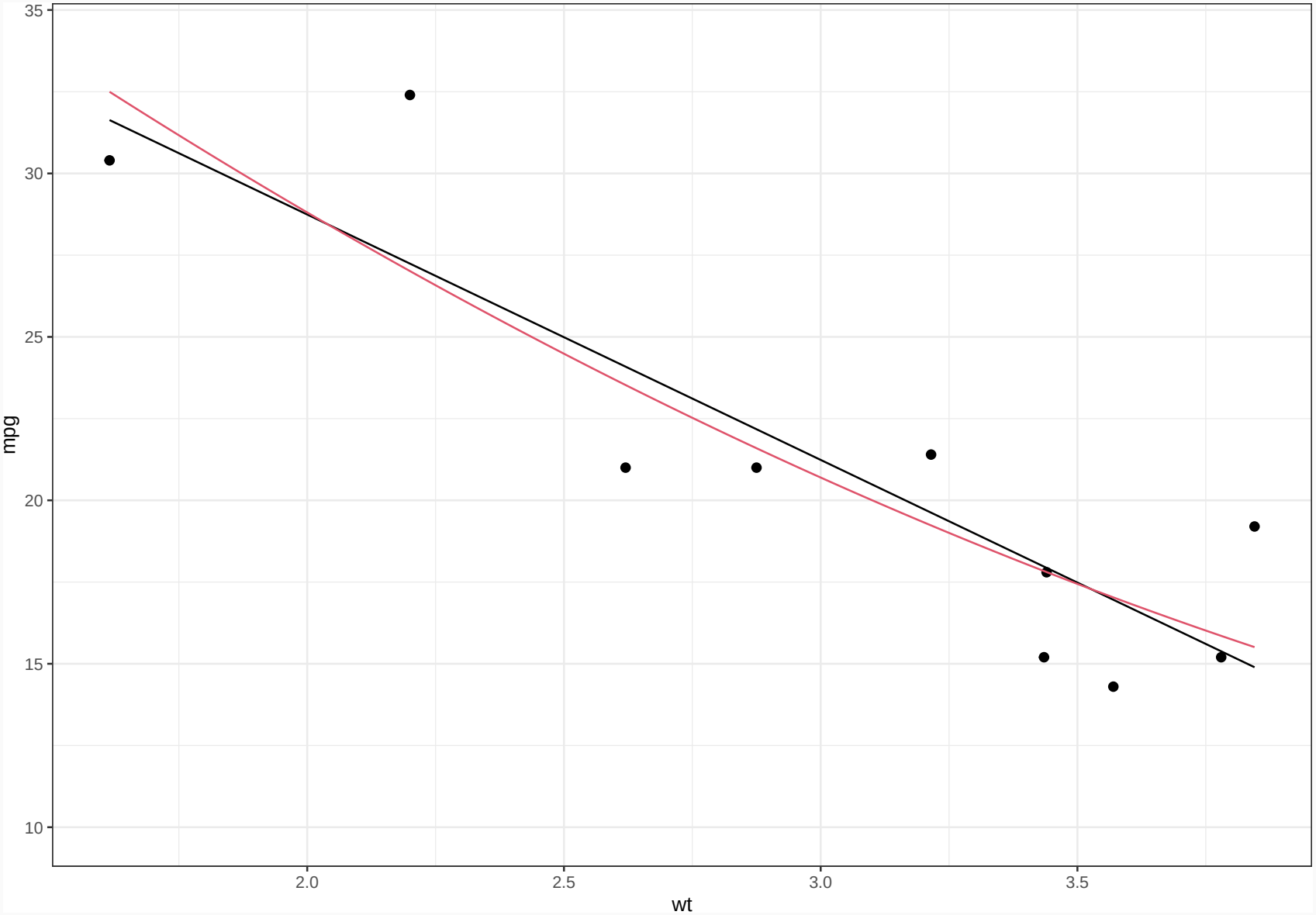
```
summary(m6)$r.squared
```

```
[1] 0.9876451
```

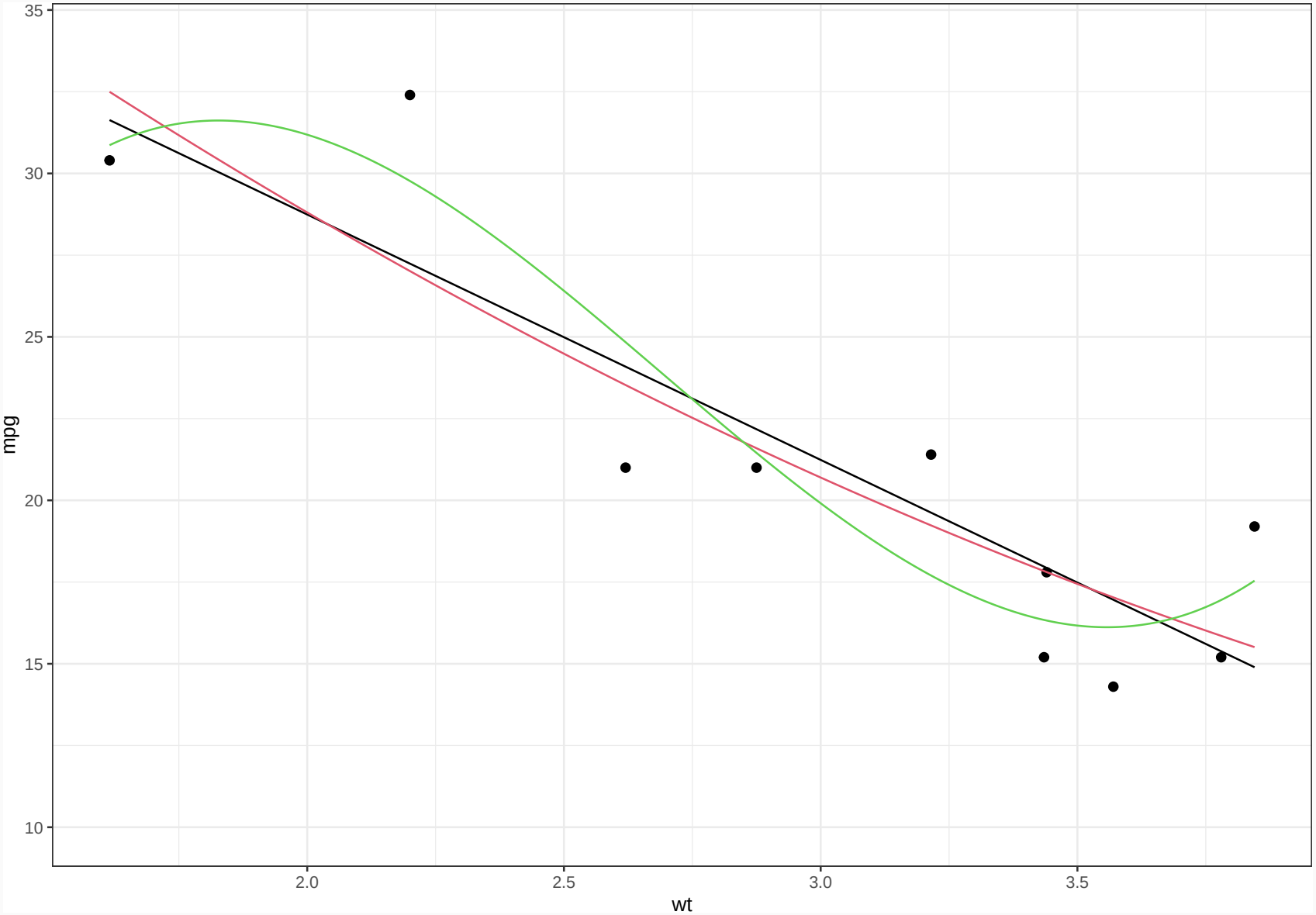
Compare these fits



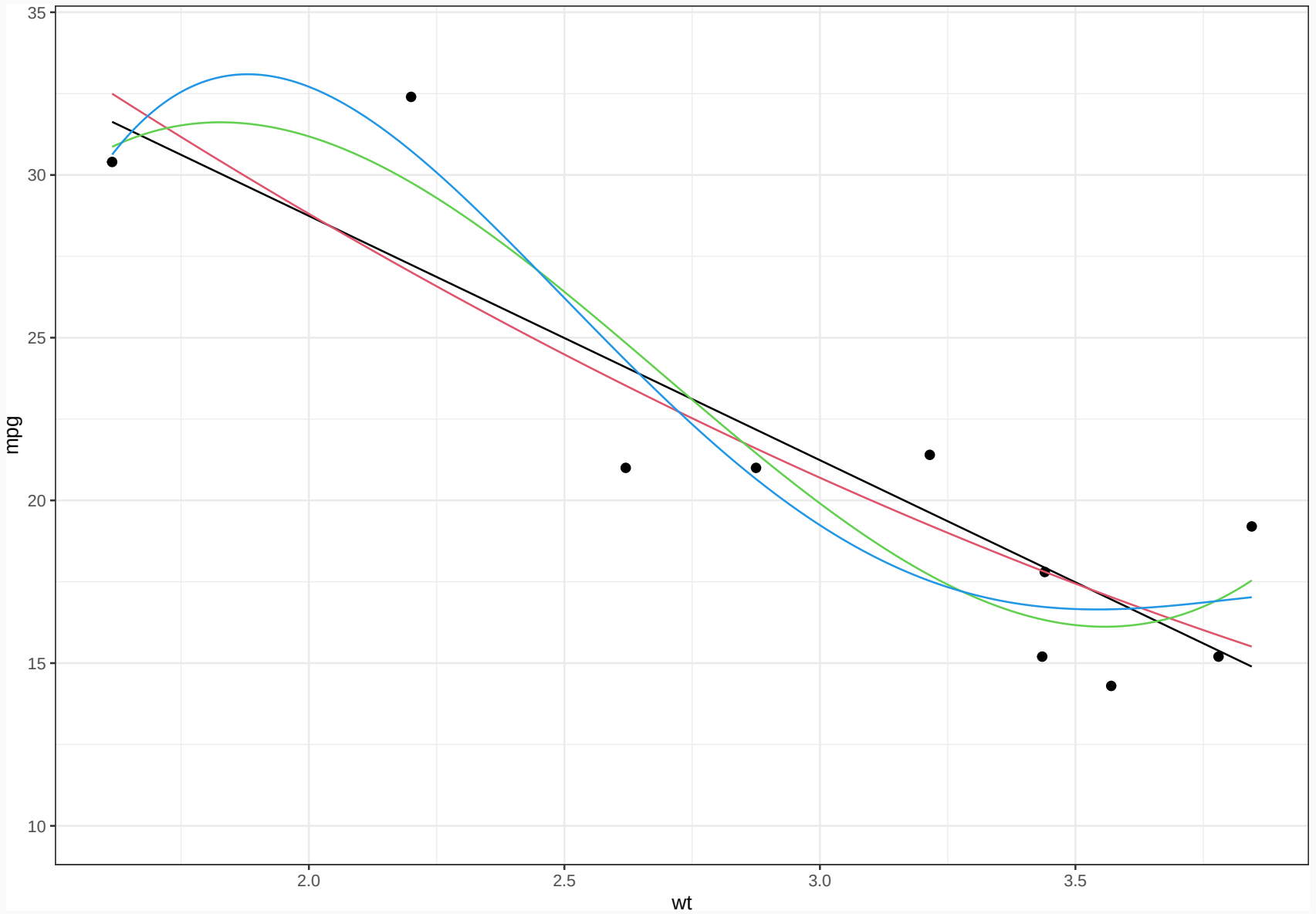
Compare these fits



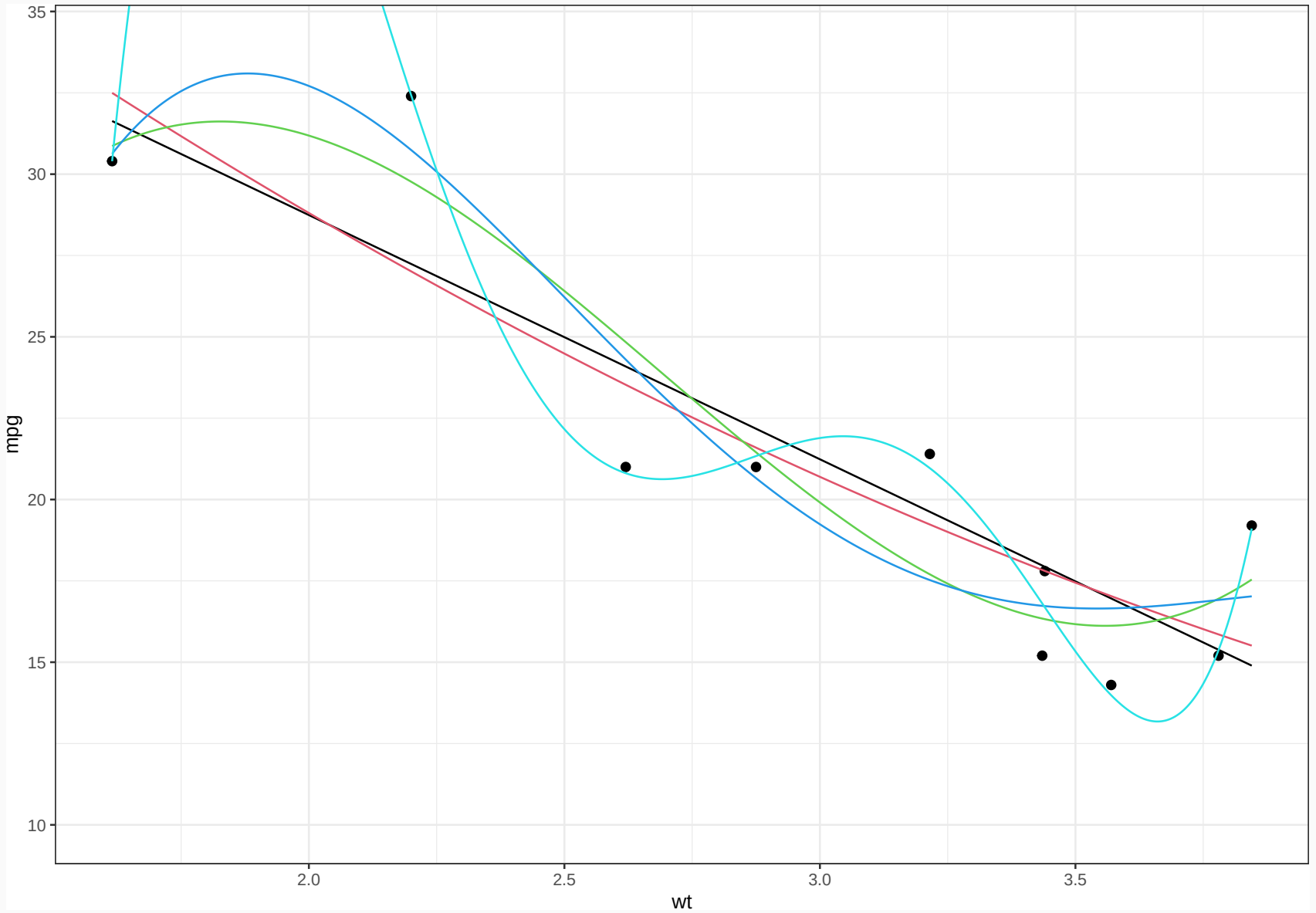
Compare these fits



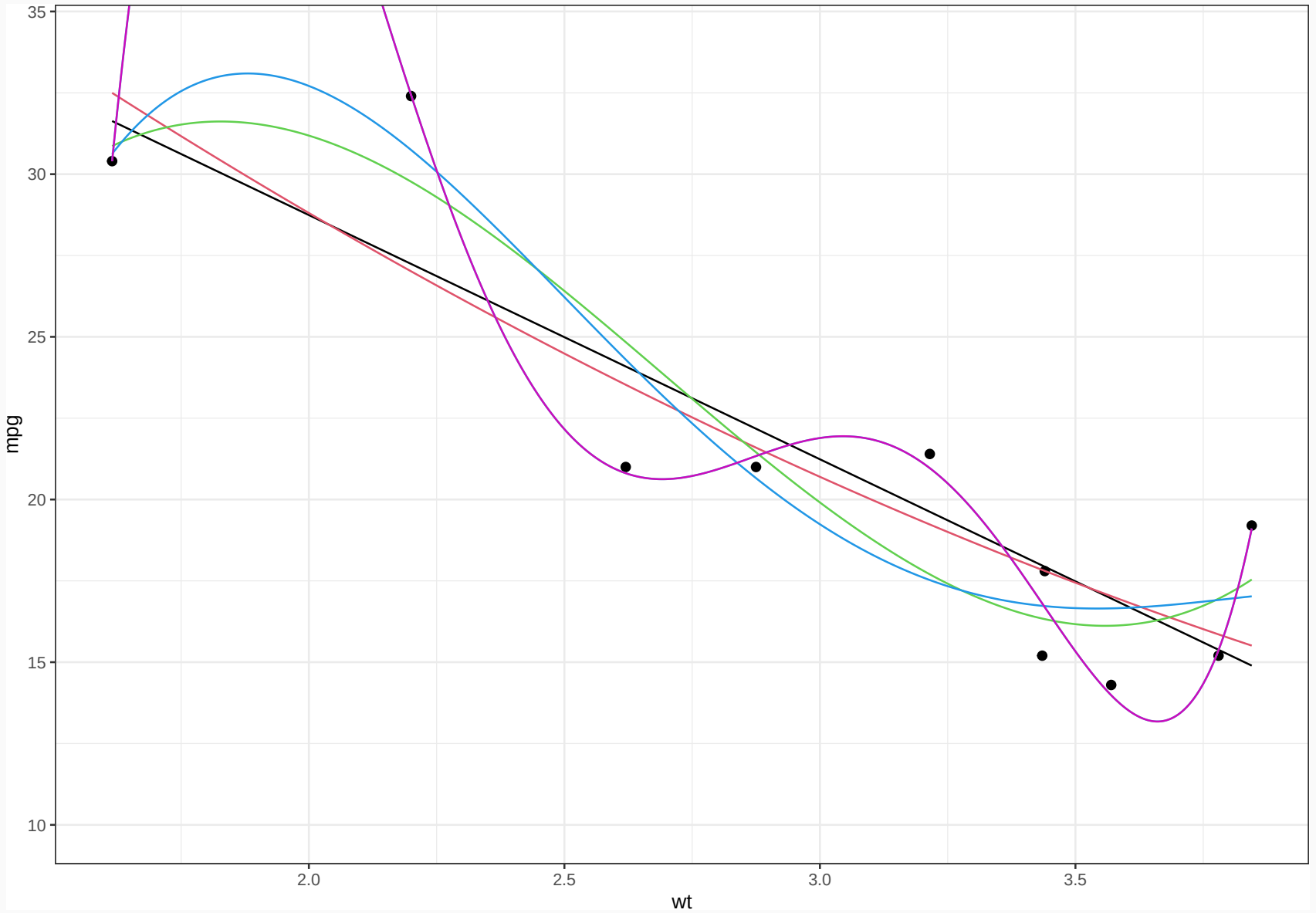
Compare these fits



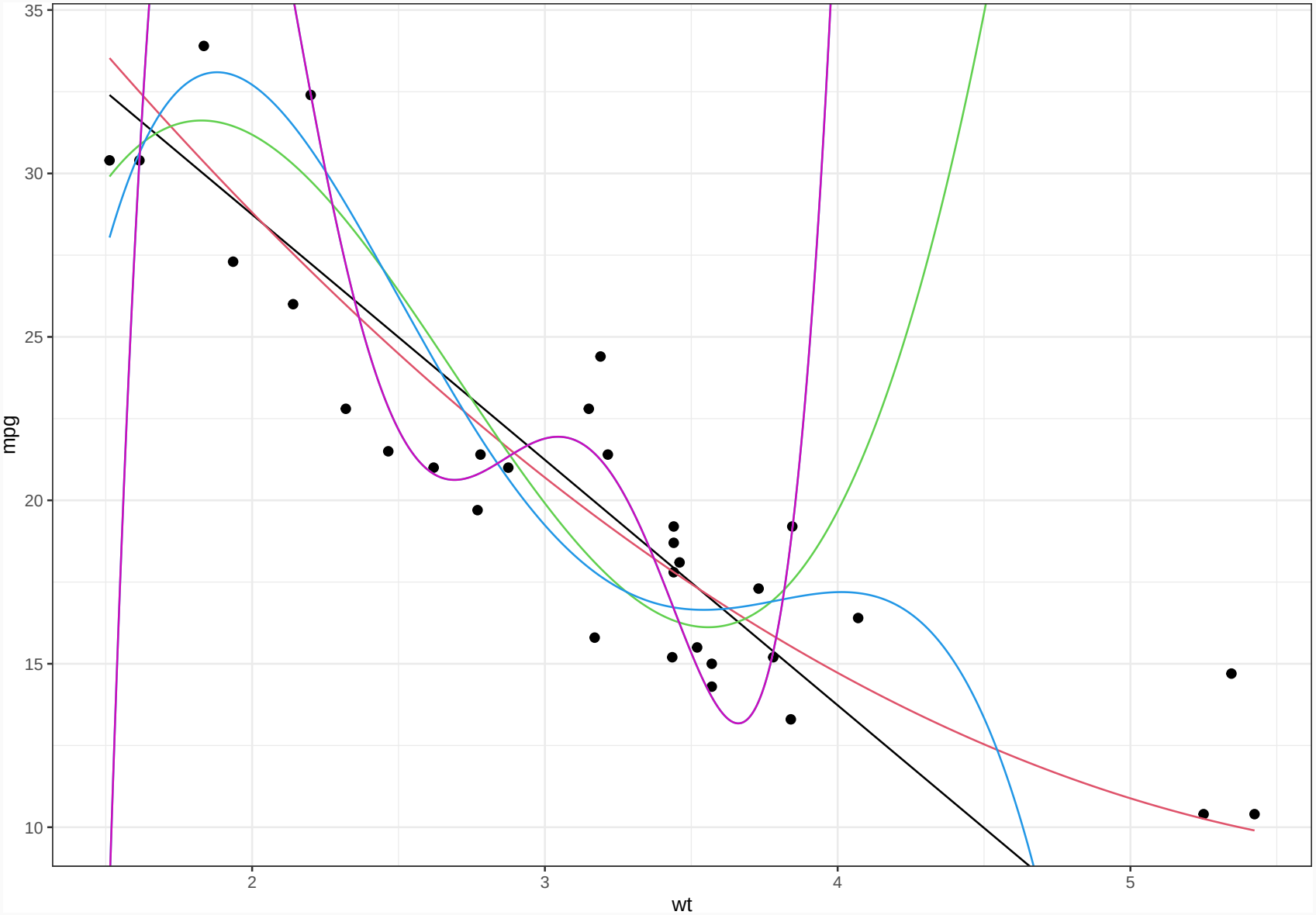
Compare these fits



Compare these fits



Compare on out-of-sample data



Overfitting and underfitting

- Adding complexity will improve goodness-of-fit measures like R^2

Overfitting and underfitting

- Adding complexity will improve goodness-of-fit measures like R^2
- But goodness-of-fit doesn't mean that we've modeled the process well, just that we've fit our *sample* well

Overfitting and underfitting

- Adding complexity will improve goodness-of-fit measures like R^2
- But goodness-of-fit doesn't mean that we've modeled the process well, just that we've fit our *sample* well
- Adding too few parameters though can mean we aren't learning enough from the data

Overfitting and underfitting

- Adding complexity will improve goodness-of-fit measures like R^2
- But goodness-of-fit doesn't mean that we've modeled the process well, just that we've fit our *sample* well
- Adding too few parameters though can mean we aren't learning enough from the data
- *Goal:* balance model complexity with predictive accuracy

Alternative fit measures

R^2 always goes up, even when we are overfitting. *Information criteria* like the Akaike Information Criterion include a penalty for model complexity. This helps protect against overfitting

```
AIC(m1, m2, m3, m4, m5, m6)
```

```
   df      AIC
m1  3 54.60078
m2  4 56.29884
m3  5 54.14155
m4  6 55.53999
m5  7 33.91024
m6  8 35.81324
```

Lower scores means a better performing model. But as we can see here, it's no guarantee...

AIC with a more realistic set of models

AIC has two components, the *log-likelihood* of the model, which tells us how well the model explains the data, and a *penalty* term for the number of parameters estimated in the regression.

```
m1 <- lm(income ~ age + gender + edu, data = dat)
m2 <- lm(income ~ age + gender + edu + hrs_work, data = dat)
m3 <- lm(income ~ age + gender * edu * hrs_work, data = dat)
m4 <- lm(income ~ age * gender * edu * hrs_work, data = dat)
m5 <- lm(income ~ (age + I(age^2)) * gender * edu * hrs_work, data = dat)
m6 <- lm(income ~ (age + I(age^2) + I(age^3)) * gender * edu * hrs_work, data = dat)
```

If two models have a similar fit to the data, AIC will tell us to prefer the less complex model.

AIC in action

Lower AIC values indicate a better fit.

```
AIC(m1, m2, m3, m4, m5, m6) |>
    arrange(AIC)
```

```
   df      AIC
m4 25 20577.74
m3 14 20583.57
m5 37 20586.82
m6 49 20589.58
m2  7 20603.32
m1  6 20657.07
```

Compared to r squared

```
summary(m3)$r.squared
```

```
[1] 0.3013753
```

```
summary(m4)$r.squared
```

```
[1] 0.3240637
```

```
summary(m5)$r.squared
```

```
[1] 0.3359174
```

```
summary(m6)$r.squared
```

```
[1] 0.3524448
```

Out of sample prediction

The gold standard for assessing the fit of a predictive model is *out-of-sample accuracy*.

Out of sample prediction

The gold standard for assessing the fit of a predictive model is *out-of-sample accuracy*.

How well does the model predict new observations?

Dividing the data into a ‘training’ and ‘test’ set

Let’s hold out a portion of the income data to evaluate model accuracy.

```
# sample 90% of rows for training the model
rows <- 1:nrow(dat)
test_rows <- sample(rows, 0.9 * nrow(dat))
# subset to sampled rows for the training data
dat_training <- dat[test_rows, ]
# and the remaineder for test
dat_test <- dat[-test_rows, ]
```

Training the models

Re-fit each model m1-m6 with *ONLY* the training data

```
m1_t <- lm(income ~ age + gender + edu, data = dat_training)
m2_t <- lm(income ~ age + gender + edu + hrs_work, data = dat_training)
m3_t <- lm(income ~ age + gender * edu * hrs_work, data = dat_training)
m4_t <- lm(income ~ age * gender * edu * hrs_work, data = dat_training)
m5_t <- lm(income ~ (age + I(age^2)) * gender * edu * hrs_work, data = dat_training)
m6_t <- lm(income ~ (age + I(age^2) + I(age^3)) * gender * edu * hrs_work, data = dat_training)
```

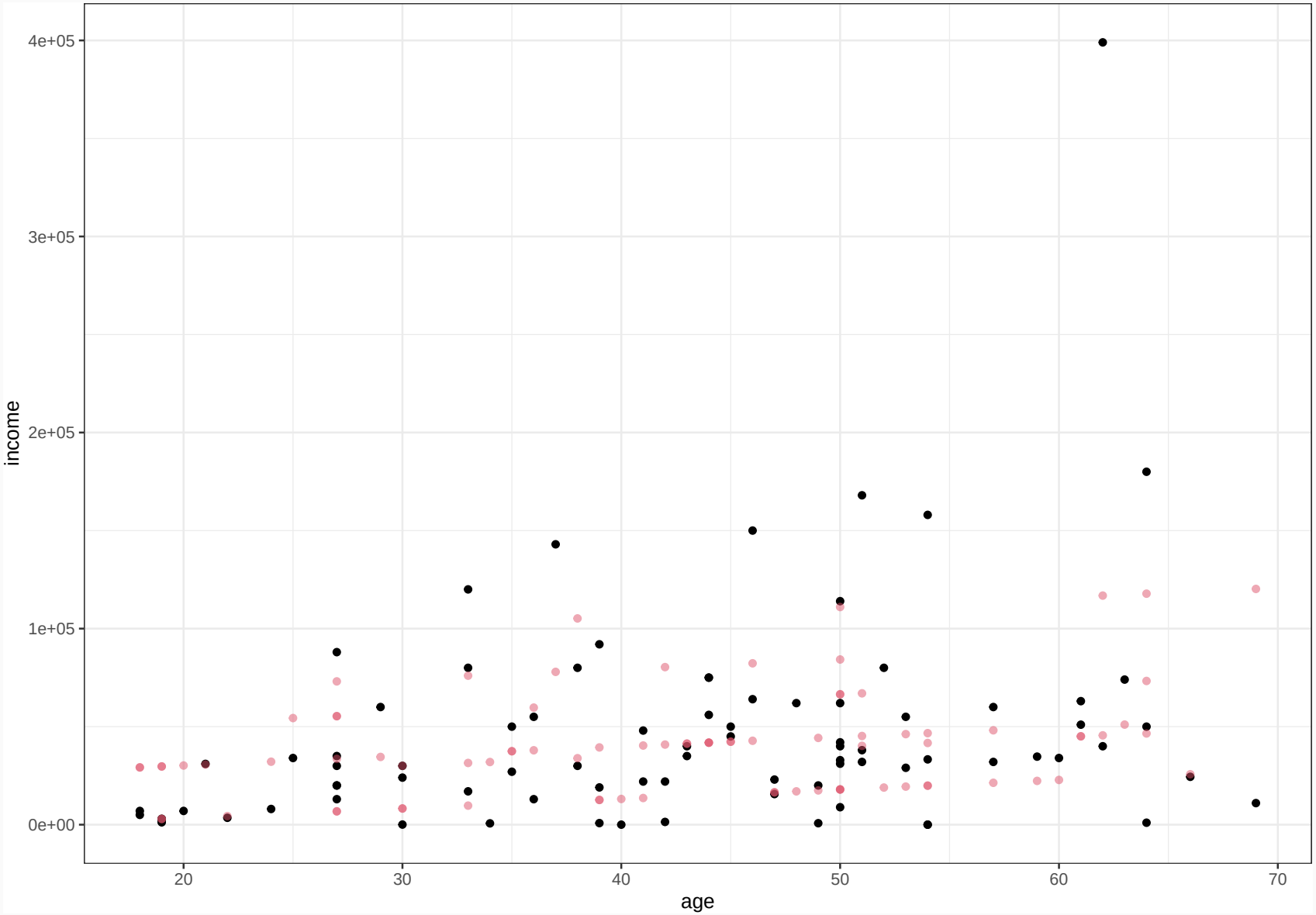
Now let’s use our models to predict

We’ll now use each model to predict income for our test data

```
m1_test <- predict(m1_t, newdata = dat_test)
m2_test <- predict(m2_t, newdata = dat_test)
m3_test <- predict(m3_t, newdata = dat_test)
m4_test <- predict(m4_t, newdata = dat_test)
m5_test <- predict(m5_t, newdata = dat_test)
m6_test <- predict(m6_t, newdata = dat_test)
```

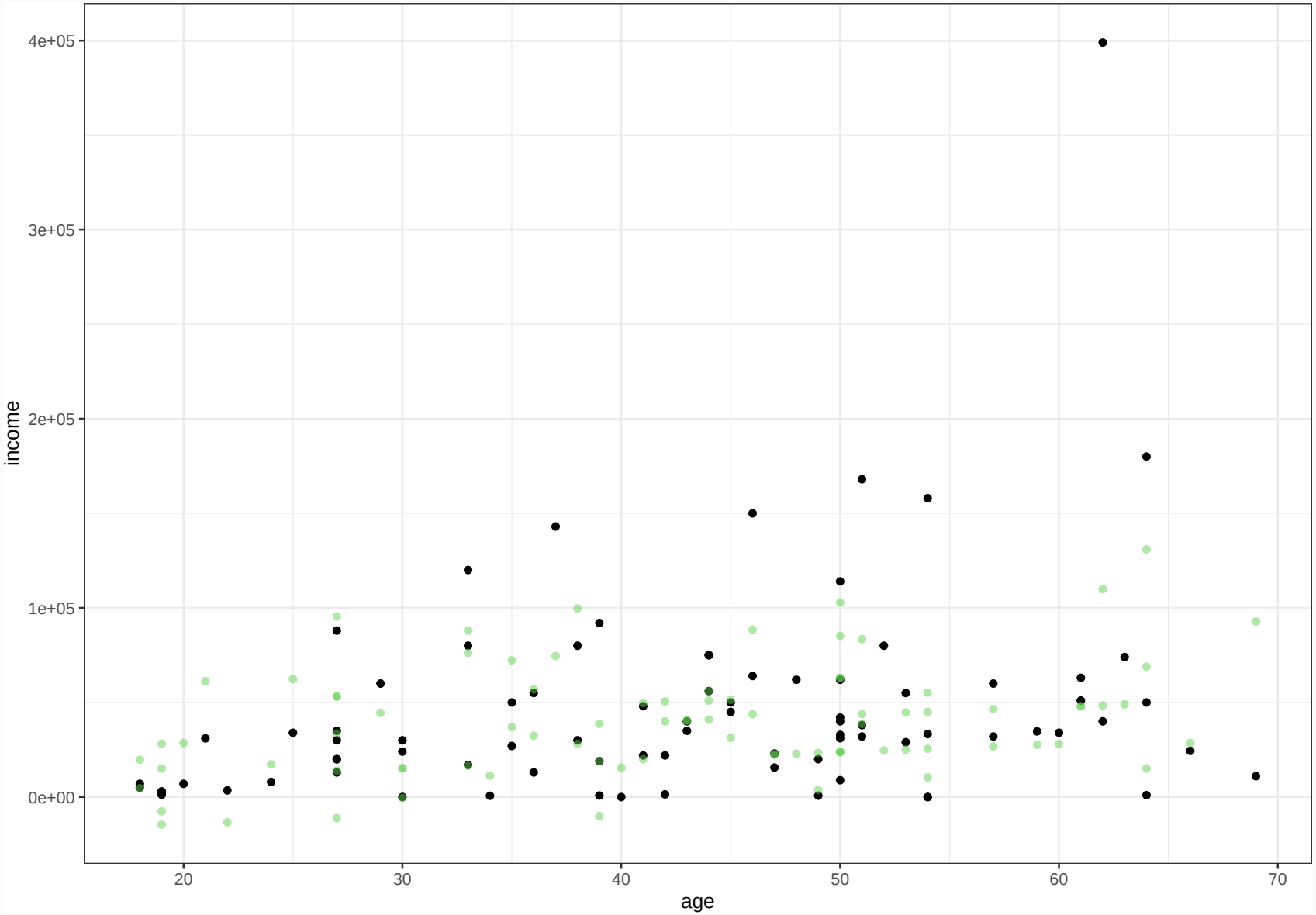
Visualize the fits

Model 1



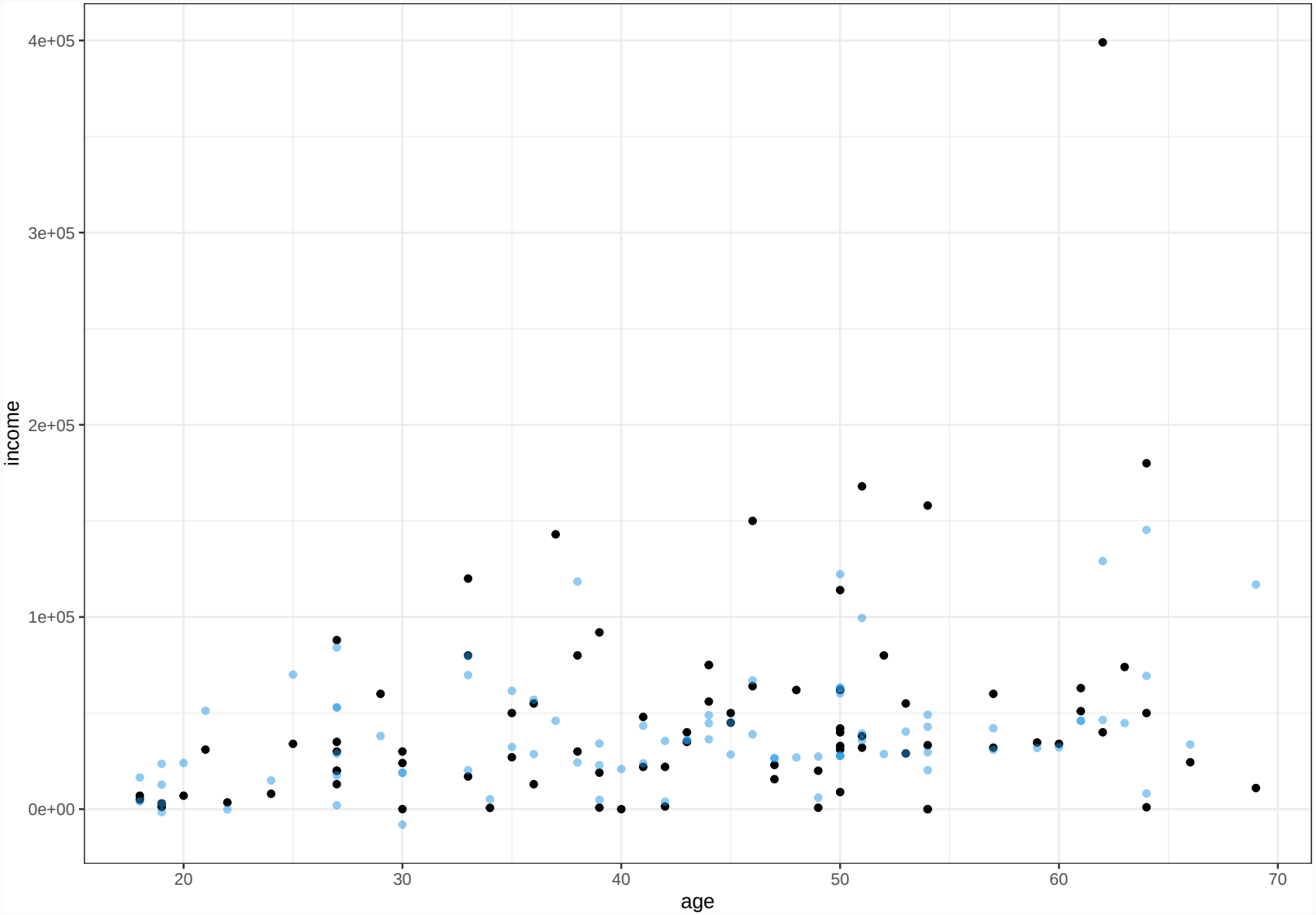
Visualize the fits

Model 2



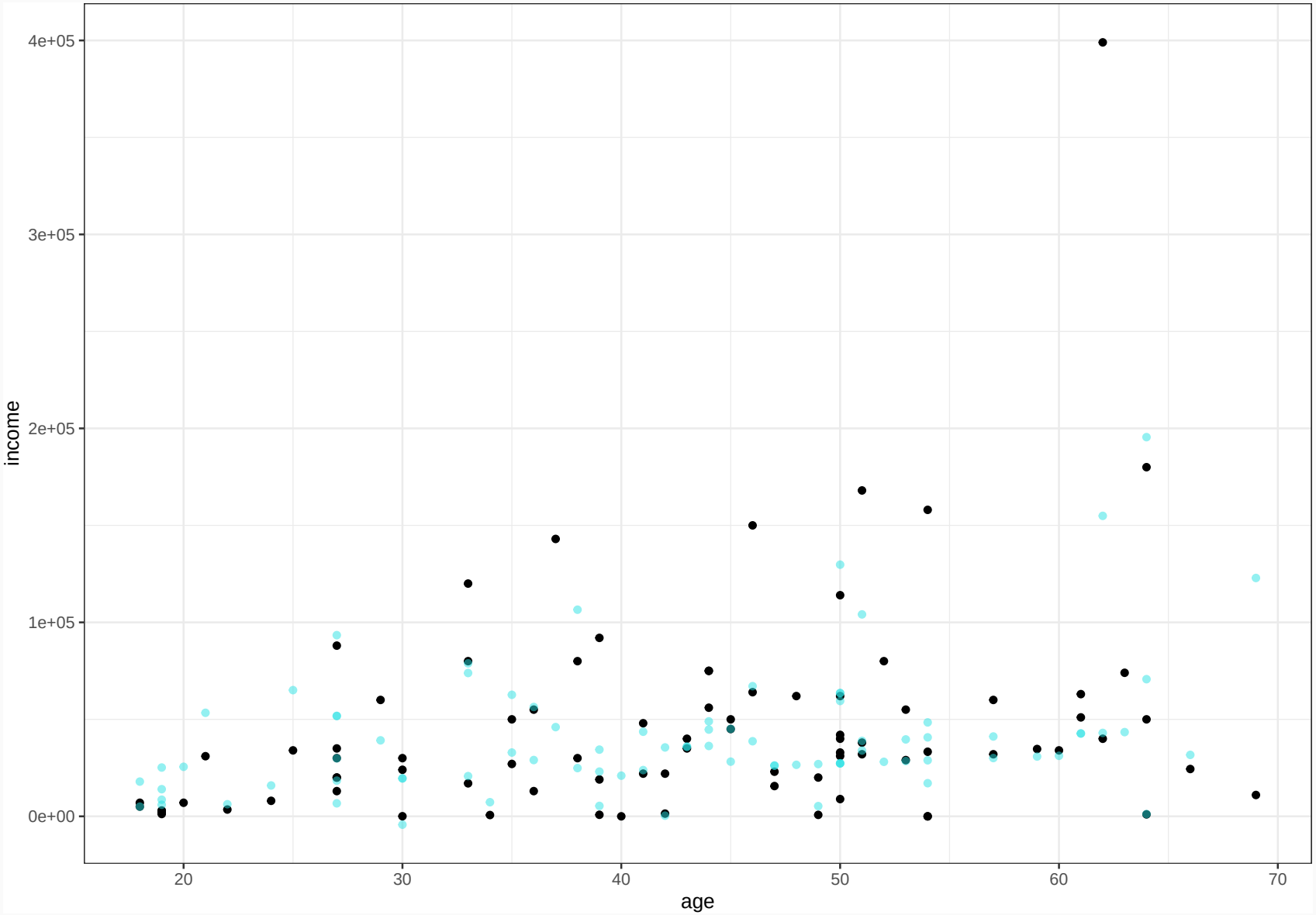
Visualize the fits

Model 3



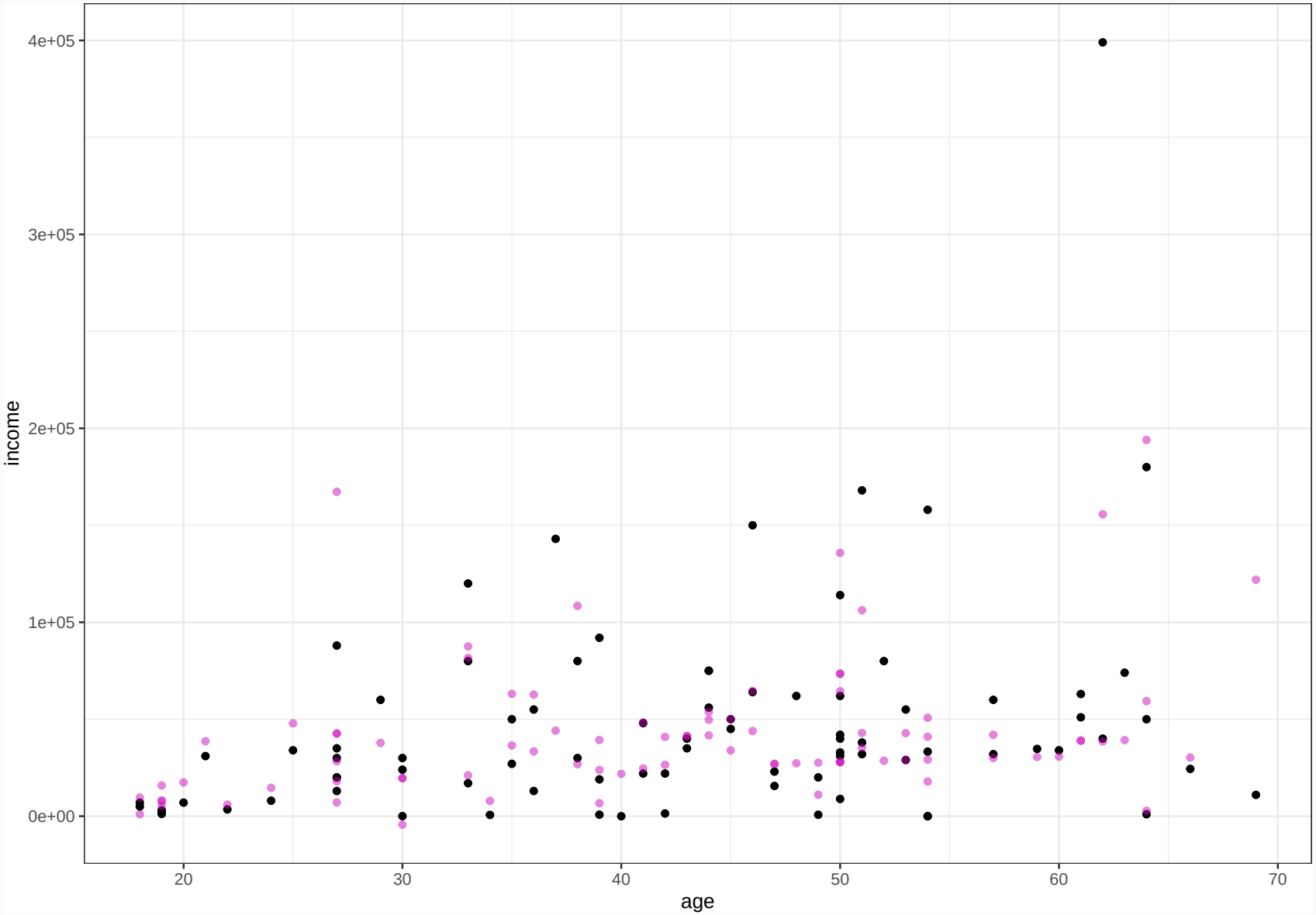
Visualize the fits

Model 4



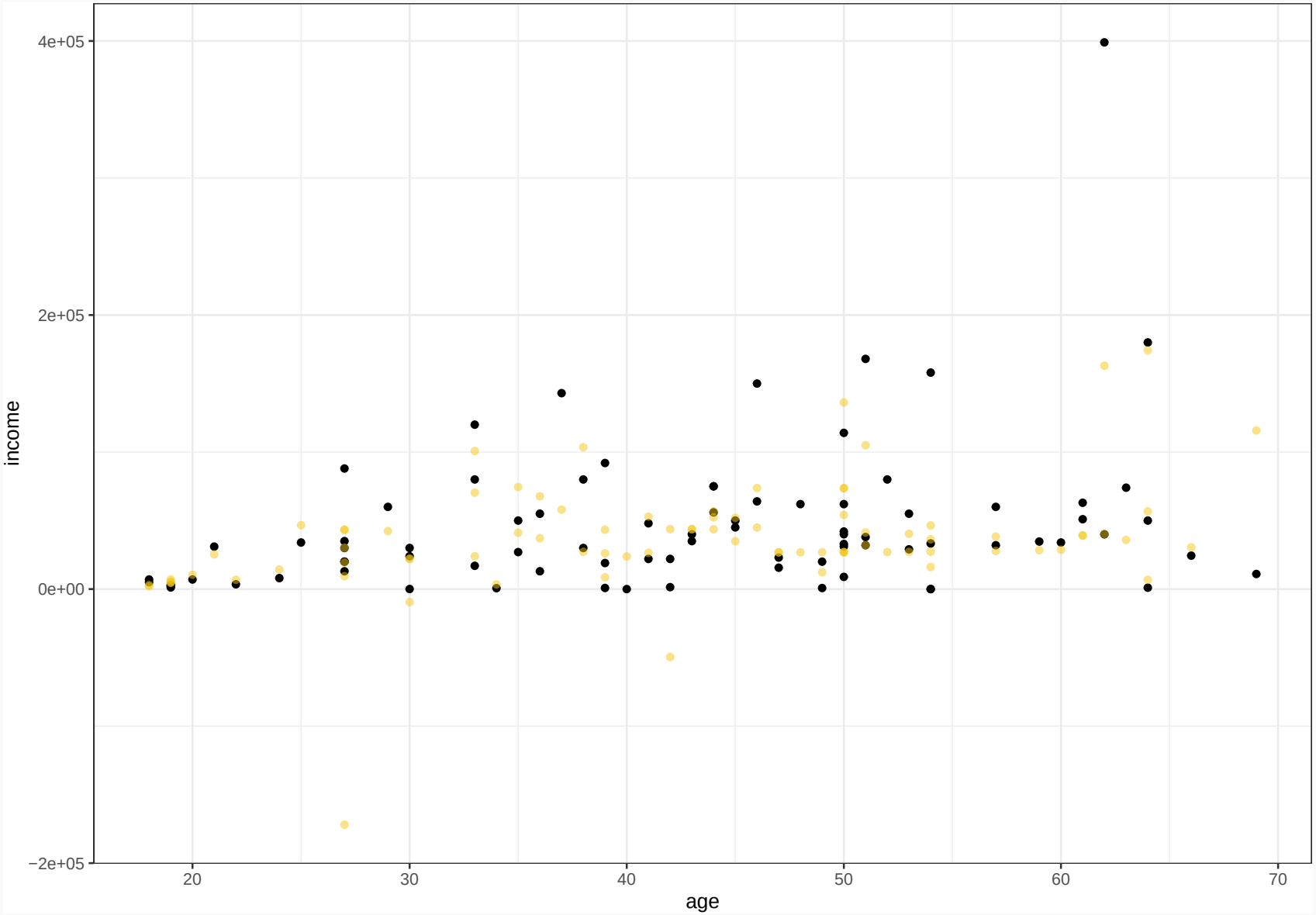
Visualize the fits

Model 5



Visualize the fits

Model 6



So which is best?

Let's compute the mean squared error to compare. Recall that error is just $\hat{y} - y$

```
resid1 <- m1_test - dat_test$income
resid2 <- m2_test - dat_test$income
resid3 <- m3_test - dat_test$income
resid4 <- m4_test - dat_test$income
resid5 <- m5_test - dat_test$income
resid6 <- m6_test - dat_test$income
```

So which is best?

We'll compare models by computing the mean squared error (and take the square root so they aren't quite so big). This is called *root mean squared error* and is a common accuracy statistic

$$RMSE = \sqrt{E((\hat{y} - y)^2)}$$

So which is best?

```
sqrt(mean(resid1^2))
```

```
[1] 47559.33
```

```
sqrt(mean(resid2^2))
```

```
[1] 44600.57
```

```
sqrt(mean(resid3^2))
```

```
[1] 44134.97
```

```
sqrt(mean(resid4^2))
```

```
[1] 42613.92
```

```
sqrt(mean(resid5^2))
```

```
[1] 44340.44
```

```
sqrt(mean(resid6^2))
```

```
[1] 46600.08
```

Out of sample validation: the recipe

1. Subset the data into training and test sets
2. Fit each model you are considering on the training data
3. Use each model to generate expected values for the test data
4. Compute root mean squared error for each
5. Select the model with the least error